

Lab 6

x86-64 Assembly: We Need to Cook

Why is a syscall? (1/2)

The idea

- A **syscall** is you asking the OS to do something you can't do yourself — read a key press, write to the screen, end the program.

The mechanism

- put the **syscall number** in `rax`
- put the arguments in `rdi, rsi, rdx, r10, r8, r9`
- run the `syscall` instruction
- the **return value** comes back in `rax`

Note: a syscall's registers mean different things for each syscall.

What is a syscall? (2/2)

A few we use this lab

`read` rax = 0
rdi=fd rsi=buf rdx=count → bytes read

`write` rax = 1
rdi=fd rsi=buf rdx=count → bytes written

`exit` rax = 60
rdi=status → never returns

```
write to screen
mov  rax, 1       ; write
mov  rdi, 1       ; stdout
lea  rsi, [msg]
mov  rdx, 13      ; length
syscall
```

Undivided signision on x86

- There is **no** “divide register by register” instruction.
- **`div r`** divides the 128-bit pair **`rdx:rax`** by **`r`**.
- quotient → **rax** remainder → **rdx**

So to divide a plain 64-bit number:

```
divide n by 10
mov  rax, n        ; dividend (low 64 bits)
xor  edx, edx      ; ZERO the high half!
mov  rcx, 10
div  rcx           ; rax = n/10 , rdx = n%10
```

Forget the **`xor edx,edx`** and you divide by a garbage 128-bit number — wrong answer or a crash.

atoi — text into a number

read gives you bytes, not a number. Walk left to right: **`result = result*10 + digit`**



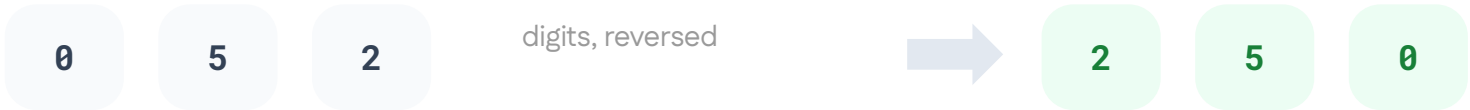

```
building 25
result = 0
'2'  ->  result = 0*10 + 2  = 2
'5'  ->  result = 2*10 + 5  = 25
'\n' ->  not a digit, stop
```

digit value of a char **`c`** is **`c - '0'`**

itoa — a number into text

Divide by 10 repeatedly; each remainder is one digit — **least significant first**.

```
pulling digits off 250
250 % 10 = 0     250 / 10 = 25
 25 % 10 = 5      25 / 10 = 2
  2 % 10 = 2       2 / 10 = 0   -> stop
```



so itoa fills the buffer, then flips it end-for-end.

C extern – can we link

- **`extern printf`** says “this name lives in another file”.
- The **linker** must be told to pull in **`libc`**.

```
two ways to link against libc
ld  -o prog prog.o -lc \
    --dynamic-linker /lib64/ld-linux-x86-64.so.2
gcc -no-pie -nostartfiles -o prog prog.o   <- the makefile
```

- **`-nostartfiles`**: “I bring my own **`_start`**”
- **`-no-pie`**: fixed addresses, so **`lea rdi, [msg]`** still works
- libc **`exit`** is not syscall 60 · align **`rsp`** to 16 before any **`call`**

C extern – can we link

- Follows the syscall convention of rdi, rsi, rdx ....
- al holds number of extra floating point arguments that were given (just clear it by xor rax, rax if not in use)
- rsp needs to be 16 byte aligned while calling the function

```
Write both of this on top of the file telling that your using this
extern printf
extern exit
section .rodata
    fmt: db "sum = %ld", 10, 0
Make sure at start rsp is aligned 16 bytes
_start:
and rsp, -16


somewhere in the middle of the file when running code
lea rdi, [fmt] first argument is the format
mov rsi, rax second argument being the number to print itself
xor rax, rax now clearing rax out for the extra vector arguments
call printf

xor rdi, rdi again exiting with code 0
call exit
```

Week 5 Slides

XMM Registers

These are special 128 bit registers used for floating point arithmetic (wired up to the parts of the cpu which have capabilities of performing floating point arth)

- **`xmm0-xmm15`** are the 16 xmm registers present in 64 bit systems. (In 32 bit systems only 8 xmm registers are present)
- **`SSE Instruction`** are special instructions that are present in the ISA to perform operations with these registers.
- **`So how do we use them ?`**

```
add_cmplx:

    movsd xmm0, [rdi+8]
    movsd xmm1, [rdi+16]
    addsd xmm0, [rsi+8]
    addsd xmm1, [rdi+16]
    movsd [rdx+8], xmm0
    movsd [rdx+16], xmm1
    ret
```

Basic xmm/SSE instructions

`movsd xmm, xmm/*mem` `movsd xmm/*mem, xmm`	Can move data to / from memory location or other xmm
`addsd xmm, xmm/*mem` `subsd xmm, xmm/*mem`	add / subtract, in place
`mulsd xmm, xmm/*mem` `divsd xmm, xmm/*mem`	multiply / divide in place
`movq xmm, reg/*mem` `movq reg/*mem, xmm`	Can move data to / from general purpose register



We are only using **`64 bits`** out of the **`128 bits`** width in xmm registers

Best Functions Forever <3

- **`call <label>:`** Pushes the address of the **next** instruction (also known as the return address) and jumps to the label.
- **`ret:`** Pops the return address from top of the stack and jumps back to respective address

```
main:
    call my_func
    ; execution resumes here

my_func:
    ; do work
    ret
```

Careful what you wish for

- What if we push and pop something else on the stack before doing ret?
- ret will read **something else** and it will not return :(



What is a stack?

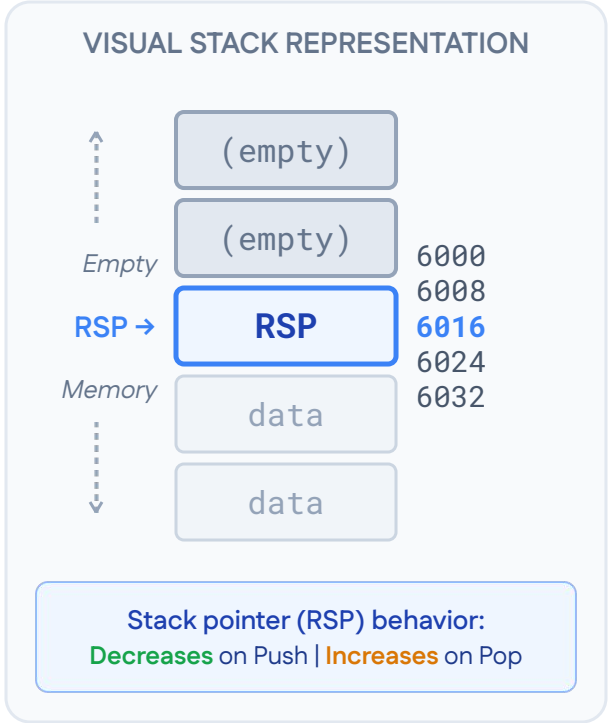
- Recall the function stack mumbo jumbo in c++
- We push things on the stack before a function call

  And pop them afterwards
- Stack grows downwards! (Higher -> lower memory addresses)
- Push operation: eg: `push rax`
- Equivalent assembly:

```
sub rsp, 8       ; say initially rsp=6016
mov [rsp], rax
```

- Pop operation: eg `pop rax`
- What should be equivalent assembly?

```
mov rax, [rsp]
add rsp, 8
```



Caller Callee Conventions

- **Caller saved registers:**
  - **Function arguments:** `rdi, rsi, rdx, rcx,r8, r9`
  - **Return value:** `rax`
  - **Scratch:** `r10, r11`
  - **All xmm registers:** `xmm0-xmm15`
- **Callee saved registers:**
  - **General Purpose Registers:** `rbx, rsp, rbp, r12, r13, r14, r15`



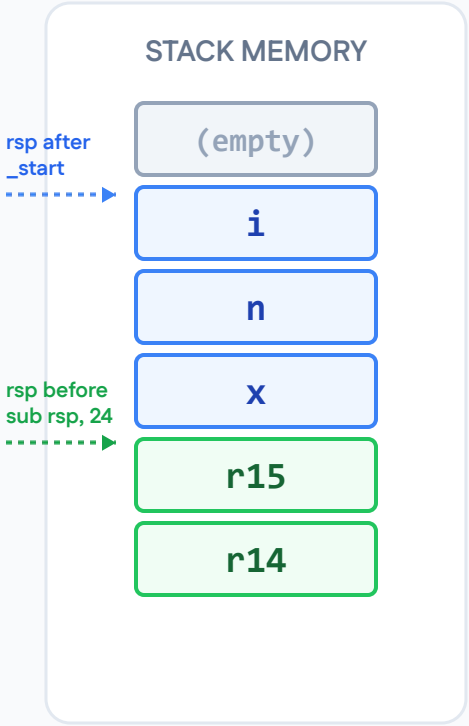
Example: Function

```
_start:
    push rbp
    push rsp
    push rbx
    push r12
    push r13
    push r14
    push r15

    sub rsp, 24

    mov rax, 0
    mov [rsp + 16], rax ; x = 0
    mov rax, 3
    mov [rsp + 8], rax ; n = 3

    mov rax, 0
    mov [rsp], rax ; i = 0
```



main.c

```
// Equivalent in C
long long x = 0;
long long n = 0;
long long i = 0;
```

Alignment is key

```
void example() {
    char a = 'A';  // 1 byte  (offset -1)
                   // 3 bytes of PADDING added here
    int b = 42;    // 4 bytes (offset -8, must be 4-byte aligned)
    char c = 'C';  // 1 byte  (offset -9)
                   // 7 bytes of PADDING added here
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)
}
```

- Recall memory alignment of data types from the theory course
- TLDR: **`sizeof(T) % alignof(T) == 0`**
- This is a standard practice (called SysV ABI) that C/C++ compilers implement for us
- In x86 we will have to manually insert “padding” to achieve this
- **Padding** – leaving some gaps in the memory address space

Alignment is key

```
void example() {
    char a = 'A';  // 1 byte  (offset -1)
                   // 3 bytes of PADDING added here
    int b = 42;    // 4 bytes (offset -8, must be 4-byte aligned)
    char c = 'C';  // 1 byte  (offset -9)
                   // 7 bytes of PADDING added here
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)
}
```

- For eg: for `int b` -> starting address should be ≡0 mod 4
- We insert padding suitable for this
- Remember xmm? Address should be 8 byte aligned while using `movsd` and `movaps/movapd` etc. Note: `movups` allows unaligned!

- When you call a function make sure its **16 byte aligned (at `call` boundary)**
- Especially when you use C functions like `printf` etc.. Why?

General functions and their convention

Register Arguments (Passed in Order)

1st Arg	2nd Arg	3rd Arg	4th Arg	5th Arg	6th Arg
`rdi`	`rsi`	`rdx`	`rcx`	`r8`	`r9`

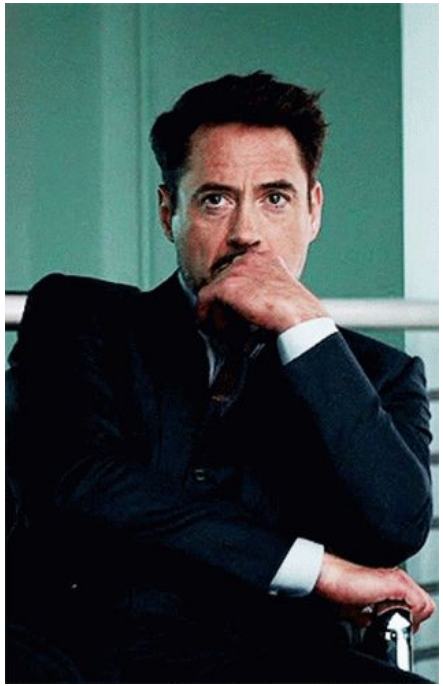
RETURN VALUE REGISTER

`rax`

Random questions

Common Doubts

- Why follow caller-callee convention?
- Why push/pop registers before & after every call?
- Why follow argument and return value conventions at all?



The Answer

MODULARITY

- Allows seamless integration with C function libraries.
- Enables using functions written by other people.
- Required to perform standard system calls.
- Mutual convention ensures code interoperability.

How do we use **C functions** in assembly?? (CHECK OUT OUTLAB)

Syscall me on my cell phone

What is a Syscall?

Requesting a privileged action from the OS (like writing to stdout).

The Mechanism

1. Load the syscall ID into `rax` (e.g., 1 for sys_write).
2. Load arguments using standard ABI rules (`rdi`, `rsi`, `rdx`).
3. Substitute `r10` in place of `rcx` if used
4. Execute the **`syscall`** instruction.

Position · Register · Description (sys_write Example)

Syscall Number `rax` (1 for `sys_write`)

1st Argument `rdi` (File Descriptor (`1` for `stdout`) for write)

2nd Argument `rsi` (Pointer to buffer string (`msg`) for write)

3rd Argument `rdx` (Length of buffer (`msglen`) for write)

**Note: Register arguments (rdi, rsi, rdx, etc.) hold entirely different meanings depending on the specific syscall invoked.*

Caller callee convention done well

- Following are some slides to give a better understanding of caller callee convention
- Optional resources:
  - Calling convention (wiki)
  - A good outline of calling convention
  - (manual) the full sysv abi which goes way beyond just caller-callee register convention

- In short, registers are limited and common for any piece of code running on the CPU. This is a set of conventions for where and how to save each register.

Good old caller-callee

- A simple example of good old caller callee convention is on the repo here

CALLER – precall and postcall work

```
_start:
    ; GOAL: compute adder(10, 20) * 3 - 10

    mov rdi, 10 ; rdi - caller saved
    mov rbx, 3  ; rbx - callee saved

    ; PRE-CALL WORK : push caller-saved regs - rax,rdi,rsi,rdx,rcx,r8-11,xmm*
    ; Q1: Why am I not pushing rax? See below at NOTE1
    push rdi
    push rsi
    push rdx
    ; ...

    mov rdi, 10   ; put arguments in rdi, rsi...
    mov rsi, 20
    call adder    ; answer is put in rax

    ; POST-CALL WORK: pop caller-saved regs
    pop rdx
    pop rsi
    pop rdi
    ; recover old rdi, rsi, rdx values

    ; NOTE1: Does this usage give you the answer to Q1?
    imul rax, rbx ; rax *= rbx (rbx is untouched by `adder`, the callee)
    sub rax, rdi  ; rax -= rdi (rdi is pushed/popped by me , the caller)
    mov r12, rax  ; stash it away in r12, r12 is my returncode value

    ; answer (came in `rax`) should sit in `rdi`
    ; for setting returncode in syscall
    mov rdi, r12
    mov rax, 60
    syscall
```

CALLEE – prologue and epilogue

```
; receives arguments in rdi, rsi, rdx, rcx, ...
adder:
    ; PROLOGUE: push callee-saved regs - rbx,rsp,rbp,r12-15
    push rbx
    push r12

    ; do work!!
    ; here i can tamper with rbx
    mov rbx, rdi
    add rbx, rsi
    mov rax, rbx ; finally store return value in rax

    ; EPILOGUE: pop callee-saved regs - rbx,rsp,rbp,r12-15
    pop r12
    pop rbx

    ret
```

An Example with Godbolt

Simple C++ code and its assembly using the caller callee convention. [Godbolt link]

The code is on the repo as `sum.c` and `sum.s`. (Assembly is slightly modified for teaching)

- C++ code. Remember every variable is a register.
- Pause here and think which registers you should use for each variable – **caller-saved** or **callee-saved**. What are the tradeoffs?

```
1   float scale_add(float a, float b) {       // leaf
2       return a * 3.0f + b;
3   }
4
5   int sum_loop(float *arr, int n) {     // caller: has a value (total) live ACROSS the ca
6       float total = 0;
7       for (int i = 0; i < n; i++) {
8           total += scale_add(arr[i], i);   // total must survive this call
9       }
10      return total;
11  }
```

An Example with Godbolt

- Simple C++ code and its assembly using the caller callee convention. [Godbolt link]
- Callee (the leaf function code)
  - Do I really need to use `rbx`? Now imagine a world with callee-saved-only convention!

    Remember that this function is called in a **loop** multiple times

```
; ---- LEAF CALLEE: borrows rbx as scratch -> must save/restore it ----
scale_add(float, float):
        push    rbx             ; PROLOGUE: push rbx on stack
        mov     ebx, 3          ; ebx = 3   (just using rbx as scratch space)
        cvtsi2ss xmm2, ebx      ; xmm2 = 3.0
        mulss   xmm0, xmm2      ; xmm0 = a * 3.0
        addss   xmm0, xmm1      ; xmm0 += b     -> return value in xmm0
        pop     rbx             ; EPILOGUE: restore rbx
        ret
```

An Example with Godbolt

- Caller

```
; ---- CALLER ----
sum_loop(float*, int):
        xorps   xmm0, xmm0 ; total = 0.0, kept directly in xmm0
        mov     r12d, 0    ; i = 0
.loop_check:
        cmp     r12d, esi             ; i vs n (n is chilling in rsi)
        jge     .loop_end
        push    rdi                   ; PRE-CALL WORK: save 'arr'
        push    rsi                   ; PRE-CALL WORK: save 'n'
        push    r12                   ; PRE-CALL WORK: save 'i'
        sub     rsp, 8                ; PRE-CALL WORK: make room on stack for 'total'
        movss   dword ptr [rsp], xmm0 ; PRE-CALL WORK: save 'total'

        ; setup call
        movss   xmm0, dword ptr [rdi + 4*r12] ; xmm0 = arr[i]   (arg 1 for scale_add)
        cvtsi2ss xmm1, r12d                   ; xmm1 = (float)i (arg 2 for scale_add)
        call    scale_add(float, float)

        addss   xmm0, dword ptr [rsp] ; POST-CALL WORK: total += return value
        add     rsp, 8                ; POST-CALL WORK: free stack slot used for 'total'
        pop     r12                   ; POST-CALL WORK: restore 'i'
        pop     rsi                   ; POST-CALL WORK: restore 'n'
        pop     rdi                   ; POST-CALL WORK: restore 'arr'
        inc     r12d                  ; i++
        jmp     .loop_check
.loop_end:
        ret          ; total is already sitting in xmm0, nothing more to do
```

Week 4 Slides

Why Assembly?

- **`1:1 Mapping`**: Assembly is the last step before raw machine code; one line ≈ one CPU instruction with a direct bijection.
- **`Zero Abstractions`**: There are no garbage collectors, heavy runtimes, or safety nets.
- **`Hardware Control`**: You directly control each and every register and byte.
- **`Maximum Speed`**: With proper assembly, you can beat compilers in execution speed.

With great power (of the CPU) comes great responsibility (for every single bit).

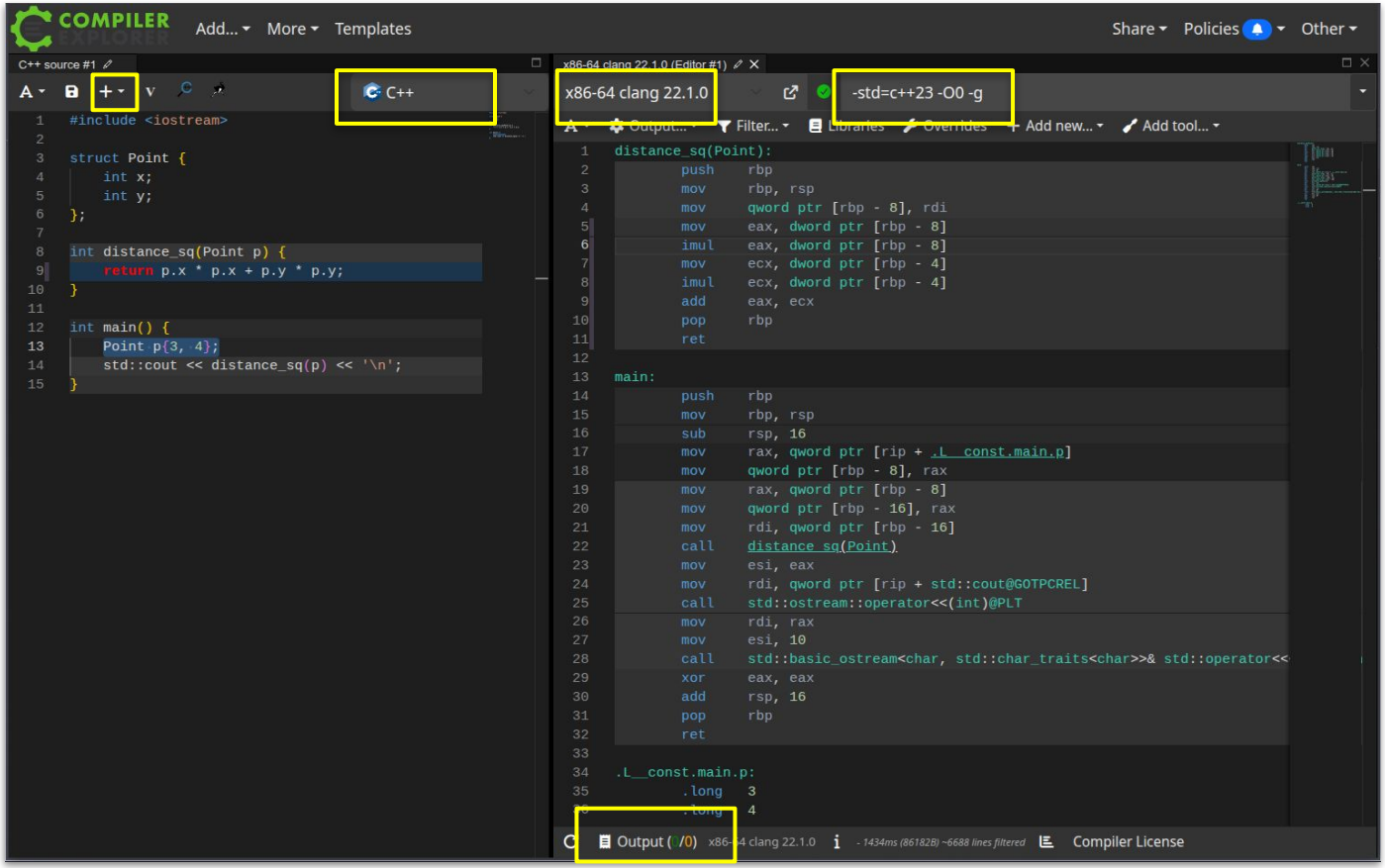
```
section .text
    global _start

_start:
    mov rax, 60 ; sys_exit
    xor rdi, rdi ; status: 0
    syscall

; Machine Code (12 bytes):
48 C7 C0 3C000000 48 31 FF 0F 05
```

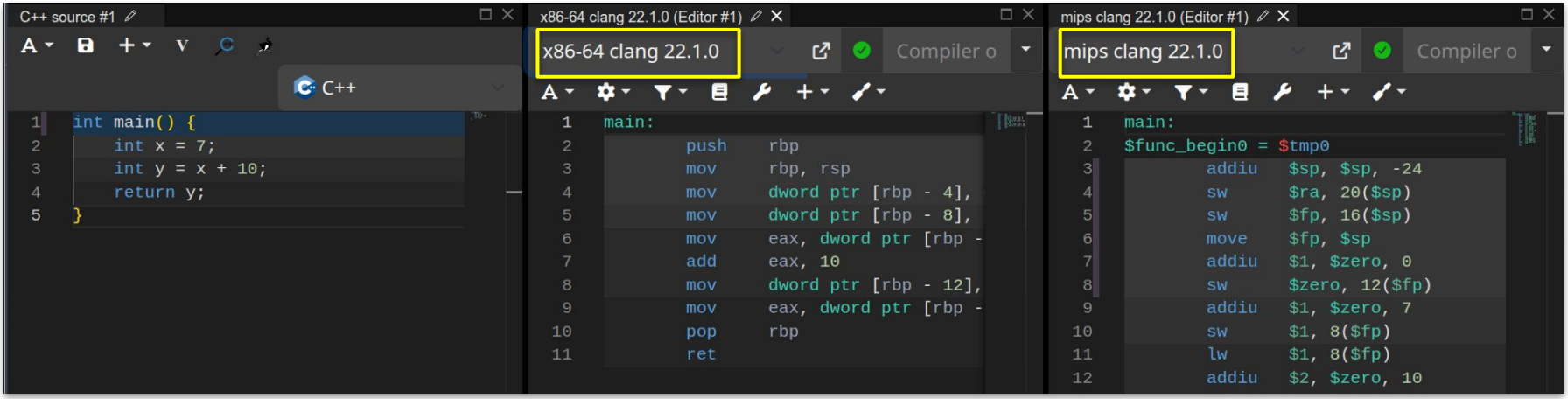
task0/exit.s

Godbolt it


COMPILER EXPLORER
Add...
More
Templates
Share
Policies
Other
C++ source #1
C++
x86-64 clang 22.1.0
-std=c++23 -O0 -g
Filter...
Add new...
Add tool...
#include <iostream>
struct Point {
int x;
int y;
};
int distance_sq(Point p) {
return p.x * p.x + p.y * p.y;
}
int main() {
Point p{3, 4};
std::cout << distance_sq(p) << '\n';
}
distance_sq(Point):
push rbp
mov rbp, rsp
mov qword ptr [rbp - 8], rdi
mov eax, dword ptr [rbp - 8]
imul eax, dword ptr [rbp - 8]
mov ecx, dword ptr [rbp - 4]
imul ecx, dword ptr [rbp - 4]
add eax, ecx
pop rbp
ret
main:
push rbp
mov rbp, rsp
sub rsp, 16
mov rax, qword ptr [rip + .L__const.main.p]
mov qword ptr [rbp - 8], rax
mov rax, qword ptr [rbp - 8]
mov qword ptr [rbp - 16], rax
mov rdi, qword ptr [rbp - 16]
call distance_sq(Point)
mov esi, eax
mov rdi, qword ptr [rip + std::cout@GOTPCREL]
call std::ostream::operator<<(int)@PLT
mov rdi, rax
mov esi, 10
call std::basic_ostream<char, std::char_traits<char>>& std::operator<<
xor eax, eax
add rsp, 16
pop rbp
ret
.L__const.main.p:
.long 3
.long 4
Output (0/0)
Compiler License

~~Compiler Explorer~~ Godbolt

- Use godbolt.org to see what your C/C++ code compiles to
- `clang` with `-O0 -g` gives nice readable code
- Click on **Output** at the bottom to see compiler stderr/stdout
- Click on some C/C++ code and press **`Ctrl+F10`** to see the corresponding assembly
- Click on the **+** button to add more tabs
- You can explore various compilers and see what the compiled output looks like



Sections in Assembly

Sections group data and instructions based on their access permissions, initialization requirements etc.

- **`section .data`** initialized storage: constants, arrays, buffers that exist before the program runs.
- **`section .text`** your actual instructions live here, organized under labels.
- **`global`** makes a symbol visible to the linker, allowing external object files or the runtime environment to reference it.
- **`_start`** is a special label marking the default entry point where program execution begins.

```
section .data
    a: dq 10
    b: dq 20

section .text
    global _start

_start:
    ; assemble here
```

task0/add.s

Labels in Assembly

- **`Labels`** act as named markers in your code, representing memory addresses of instructions or data.
- They end with a colon (**`:`**) when defined, serving as targets for jumps, loops, and function calls.
- **`jmp label`** redirects execution flow straight to the labeled instruction.

- **Recall from theory:** How do you track which instruction is being executed? What does a “jump” mean then?

- Can a label be used for a non-jump instruction?

```
section .text
global _start

_start:
; main execution point
    mov rcx, 5

.loop_start:
    dec rcx
    jnz .loop_start; jump if != 0
```

Registers: Big Potato Stuff

- **Registers** are tiny, blazing-fast storage slots built directly into the CPU taking almost zero time to access.
- **16 General-Purpose Registers**: rax, rbx, rcx, rdx, rsi, rdi, rbp, rsp, r8–r15.
- The same GPR can be accessed in many bit-widths
- rax → eax → ax → (ah, al); rdi → edi → di → dil; r8 → r8d → r8w → r8b
- **Specialized Registers** also exist for floating-point/vector operations, status etc (such as **xmm0–xmm15** for 128-bit SIMD, **ymm0–ymm15** for 256-bit SIMD, **eflags**, the status register and many more).

One Register, Several Widths (Nesting of rax)



Basic Instructions

`mov dst, src`	copy a value into a **register** or **memory location**
`add/sub dst, value`	add / subtract, in place
`imul dst, value`	signed multiply
`xor reg, reg`	the idiomatic way to zero a register
`inc/dec dst`	increment / decrement by 1
`cmp a, b`	compute a − b, discard result, but update some flags
`push/pop value`	save / restore a value on the **stack**
`lea dst, addr`	compute a memory address without loading it



What is a value? Reg? Memory? Constant/Immediate value? What is addr? – **Coming Later**

Instruction Operands & Inputs

Assembly instructions perform operations on data, but where does that data come from?

The CPU handles three distinct types of **operands** (inputs and destinations) depending on where the values are physically stored and how quickly they need to be accessed.

Registers (reg)

Blazing-fast storage locations built directly inside the CPU core. Limited in number but near-instant to access.

```
mov rdi, rax
```

mov dst_reg, src_reg

Memory (*mem)

System RAM addresses, written inside brackets. Accessing memory is much slower than registers.

```
mov rax, [rbx]
```

mov reg, [mem_addr]

Immediates (imm)

Hardcoded constant values or raw numbers embedded directly within the instruction itself.

```
mov rbx, 42
```

mov reg, constant

Crucial Hardware Constraint: You **cannot** transfer data directly from memory to memory (e.g., '`mov [rsi], [rdi]`' is illegal).

Can you taste your code?

- Exact assembly syntax – order of arguments, what operands are allowed, some fancy symbols ($,%) – depends on the **syntax flavour**

- For eg: to move value in `rbx` to `rax` ⇒
  Intel: `mov rax, rbx`
  AT&T: `movq %rbx, %rax`

  - In AT&T, some instructions (like `mov`) have suffixes that tell the CPU what operand size to use or how to do bit extension
  - E.g. `b/w/l/q` specify size; `z` indicates zero-extension, as opposed to sign-extension



- How to view disassembled output from a compiled file?
  `objdump -d -M intel file.o` → To see Intel syntax
  `objdump -d -M att file.o` → To see AT&T syntax

- Or use godbolt.org :)

Assembly follows my brother

```c
static long a = 10;
static long b = 20;

long f(long a, long b) {
    return a + b;
}

int main() {
    return f(a, b);
}
```



```asm
section .data
    a: dq 10
    b: dq 20

section .text
    global _start

f:
    mov rax, rdi
    add rax, rsi
    ret

_start:
    mov rdi, [a]
    mov rsi, [b]
    call f

    mov rdi, rax
    mov rax, 60
    syscall
```

Mommy where's my data?

```
section .data
    a:      db 10           ; 1 byte
    b:      dw 1000         ; 2 bytes
    c:      dd 100000       ; 4 bytes
    d:      dq 10000000000  ; 8 bytes
    arr:    dq 1, 2, 3, 4   ; multiple values
    msg:    db "Hello", 0   ; string

section .bss
    buffer: resb 64         ; reserve 64 bytes
    arr2:   resq 10         ; reserve 10 × 8 bytes
```

- `.data` = **initialized data**
- `.bss` = **uninitialized storage**
- `db/dw/dd/dq` define 1/2/4/8-byte values, while `resb/resw/resd/resq` reserve 1/2/4/8-byte spaces
- label (eg: `a,b,arr,msg`) allows you to access these in code

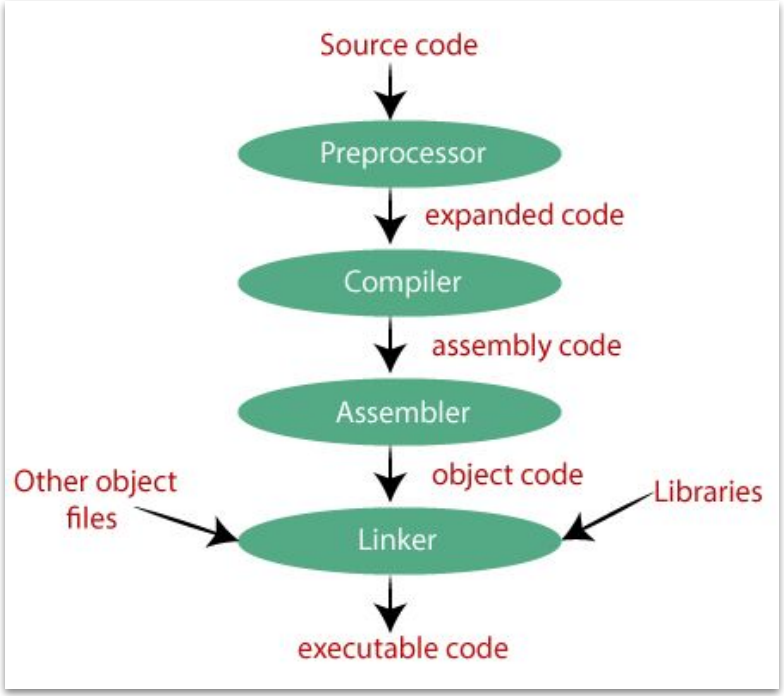
From Source Code to Executable

1. Source (.c) -> Assembly (.s)
   `$ gcc -S hello.c -o hello.s`

2. Assembly (.s) -> Obj File (.o)
   `$ nasm -f elf64 hello.s -o hello.o`
   -f specifies format (using ELF64 here)
   (Or, `objdump -d -M intel hello.o`)

3. Obj file (.o) -> Executable (.out or none)
   `$ ld hello.o main.o -o main`

- You can use **libc (standard C library)** (eg. printf etc.) in assembly by linking with it!
  - Easy: Use GCC: **`gcc hello.s -o hello`**
  - Or: Use ld: `ld hello.o -o hello -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2`



Jumps and Branching

- `cmp a, b` sets CPU flags based on (*a* − *b*). Subsequent jump instructions can use these flags to jump conditionally
- `jmp label` — unconditional, always taken.
- `je / jne` — jump if equal / not equal.
- `jl / jle / jg / jge` — signed < / ≤ / > / ≥.
- `jb / jbe / ja / jae` — unsigned < / ≤ / > / ≥.
- There's no if/else keyword — you build control flow yourself, one cmp and one jump at a time.
- An “if” in assembly is a comparison, a conditional jump past the body, and a label to land on.

```
jnc .else ; opposite of cond
.if:
; code to run if cond is true
jmp .end
.else:
; code to run when cond is false
.end:
```

Arrays

- An array is just consecutive values in `memory`
- `dq 5, -2, 8, 3, -1, 6` defines six 64-bit values, back to back.
- **`[vec1 + 8*rcx]`** reads index `rcx`; we have to multiply `rcx` by `8` as the value is in bytes and each element is 8 bytes long here.
- Get the stride wrong and you silently read wrong bytes.
- Can 8 be replaced by any arbitrary integer?

```
vec1:
    dq 5, -2, 8, 3, -1, 6
.end:
vecsize equ (vec1.end - vec1)/8

    mov rsi, [vec1+8*rcx]
```

Memory Access Modes

[base + index*scale + displacement]

– In x86-64 assembly, the processor calculates address offsets in hardware using a single, unified instruction formula: **`[base + index*scale + displacement]`**.

– This powerful addressing mode lets you locate elements in arrays, structs, or matrices in a single instruction cycle without manual shift arithmetic.

Base `[reg1]`
Any 64-bit general-purpose register pointing to the start of the structure or memory block.

Index `[reg2]`
A register holding the dynamic index (e.g., loop counter) of the element to be accessed.

Scale `[* 2^x]`
A hardware multiplier: can only be `1, 2, 4,` or `8` to match standard element widths (`byte, word, dword, qword`).

Displacement `[+ C]`
A constant immediate value (offset) added to the address. Ideal for accessing a specific field within an array of structures or skipping headers.

- Eg: `mov eax, [rbx + rcx*4 + 8]`
- Equivalent to: `eax = *(int *)(rbx + rcx*4 + 8)`

Loops

- **No `for`/`while` keyword.** A loop in assembly is just four ingredients: a **label** to jump back to, a **bound check**, a **body**, and a **jump back to the check**. You assemble these yourself, every time.
- **The four-step pattern:**
  - **1. Init:** set up your index/accumulator with your starting values
  - **2. Check:** compare against bound (repeating condition)
  - **3. Body:** do the actual work for the iteration.
  - **4. Advance:** update the index, then jump back to re-check.
- **Where you check matters.** Both of the versions to the right do the same thing, but **removing the top check** in the upper version makes it a `do {} while()` loop

```
xor rcx, rcx ; i = 0
cmp rcx, N
jge .forend
.forbegin:
    ; ...
    ; loop body
    ; ...
    inc rcx
    cmp rcx, N
    jl .forbegin
.forend:
```

```
xor rcx, rcx ; i = 0
.forbegin:
    cmp rcx, N
    jge .forend
    ; ...
    ; loop body
    ; ...
    inc rcx
    jmp .forbegin
.forend:
```

Who's the Impostor?

- Which for loop is correct/better to use?

```
.for:
    mov rsi, qword [vec1+8*rcx]
    imul rsi, qword [vec2+8*rcx]
    add rdi, rsi
    inc rcx
    cmp rcx, vecsize
    jl .for
```

```
.for:
    cmp rcx, vecsize
    jge .done

    mov rsi, [vec1 + 8*rcx]
    imul rsi, [vec2 + 8*rcx]
    add rdi, rsi

    inc rcx
    jmp .for
```

It’s Doomin’ Time

Food for thought:

1. x86 provides various operand combinations, as opposed to MIPS32 which mostly uses register–register operations. Do you think this makes x86 code faster, more convenient, both, or neither?

2. If registers are much faster to access than memory, why do CPUs use a relatively small number of registers instead of thousands of them?

3. `mov` is one of the most fundamental instructions in assembly. Do you think x86's mov alone is sufficient to express any computation?

4. (Explore) Are there x86 instructions that can do an arbitrary amount of “work”, as opposed to MIPS32 instructions which perform a fixed amount of work? What might be an example?